

Parallel and Distributed Computing

Study Guide

=== PAST PAPERS ===

Sessional-I Exam, Spring 2025

Question 1 (Gustafson's Law vs Amdahl's Law)

Q: How does Gustafson's law differ from Amdahl's law?

In Amdahl's law, we keep the workload (input data size) fixed across runs with an increasing number of processors. In Gustafson's law, we scale the workload (input data size) in proportion to the number of processors (N) to exploit larger parallel computing resources.

Question 2 (Gustafson's Law Derivation)

Q: Derive Gustafson's law using the symbols: s (serial fraction of time spent executing on parallel processor), p (parallel fraction on parallel processor), and N (number of processors). Clearly mention what each symbol means. Your final expression should be in terms of N and s only.

Let the execution time on a parallel processor be normalized to:

$$T_{\text{par}} = s + p = 1$$

Assuming there are no parallelization overheads, a single processor (hypothetical serial run) would take N times longer to complete the parallel portion. Therefore, the sequential execution time T_{seq} is:

$$T_{\text{seq}} = s + N * p$$

The scaled speedup (S) is defined as:

$$S = T_{\text{seq}} / T_{\text{par}} = (s + N * p) / (s + p)$$

Since $s + p = 1$ implies $p = 1 - s$, we substitute p into the speedup equation:

$$S = s + N(1 - s)$$

$$S = s + N - Ns$$

$$S = N + (1 - N)s$$

(where s is the serial execution time fraction on a parallel processor, N is the number of processors, and S is the scaled speedup).

Question 3 (Gustafson's Assumptions)

Q: What are the two assumptions made in the derivation of Gustafson's Law?

- We assume that each processor is equally capable and can divide and finish its allocated work at the same rate.
- We assume there are no parallelization or communication overheads, and data-based scaling will perfectly continue with an increasing number of processors.

Question 4 (Formulas)

Q: Write Amdahl's law and Karp-Flatt metric (K-F metric) formula (no derivation needed).

- **Amdahl's Law:** Speedup (S) = $1 / (F + (1-F)/N)$ (where F is the serial fraction of the code, and N is the number of processors).
- **Karp-Flatt Metric:** $f = ((1/S) - (1/N)) / (1 - (1/N))$ (where S is the experimentally achieved speedup, and N is the number of processors).

Question 5 (Comparison Table Calculation)

Q: Assume that 90% of an algorithm can be parallelized while the remaining 10% must be executed sequentially. We have partially filled the following table with 4 missing values (A, B, C, D). Calculate these four missing values:

Number of Processors used (N)	2	4	6	8
Amdahl's speedup	A (1.818)	3.077	4	4.706
Gustafson's speedup	B (1.9)	3.7	5.5	7.3
K-F metric using Amdahl's speedup	C (0.1)	0.1	0.1	0.1
K-F metric using Gustafson's speedup	D (0.053)	0.018	0.014	0.014

Value A: $S = 1 / (0.1 + 0.9/2) = 1 / 0.55 \approx 1.818$

Value B: $S = 2 + (1 - 2) * 0.1 = 2 - 0.1 = 1.9$

Value C: $f = ((1/1.818) - 1/2) / (1 - 1/2) = (0.55 - 0.5) / 0.5 = 0.1$

Value D: $f = ((1/1.9) - 1/2) / (1 - 1/2) = (0.5263 - 0.5) / 0.5 \approx 0.053$

Question 6 (K-F Interpretation under Amdahl's Speedup)

Q: Review the K-F metric derived from Amdahl's speedup in your completed table. What conclusions can you draw, and why?

The Karp-Flatt metric remains constant at 0.1 as we increase the number of processors. This is because Amdahl's law is a theoretical model that assumes a fixed problem size and zero parallelization overhead. Since K-F metric uses these theoretical speedup figures, it is unable to capture any growing overheads and simply returns the constant serial fraction (10%).

Question 7 (K-F Interpretation under Gustafson's Speedup)

Q: Review the K-F metric derived from Gustafson's speedup in your completed table. What conclusions can you draw, and why?

The Karp-Flatt metric decreases as we increase the number of processors (from 0.053 to 0.014). This is because Gustafson's law scales the problem size with the number of processors. Consequently, the parallelizable portion grows, making the sequential execution part a smaller fraction of the overall scaled workload.

Question 8 (Experimental Karp-Flatt Trend)

Q: Previously, we computed K-F metric values using theoretical speed-up figures from Amdahl's or Gustafson's law rather than measuring them directly by executing the code. For a fixed workload, would you expect such experimental K-F values to decrease, increase, or remain constant and why (only for Amdahl's case)?

We expect experimental K-F values to increase with the number of processors. Experimental speedup is lower than theoretical Amdahl's speedup due to real-world parallelization overheads (e.g., communication costs, synchronization delays, thread creation). Since the Karp-Flatt metric captures these overheads under the umbrella of "serial fraction," a rising overhead with more processors results in a rising experimental K-F metric value e .

Question 9 (Taxonomy Application)

Q: You are designing a cloud-based image processing service where each uploaded image is independently transformed (e.g., resized and enhanced). With thousands of concurrent uploads, which Flynn's Taxonomy architecture best supports parallelism and scalability? Just name it and provide rationale.

MIMD (Multiple Instruction stream, Multiple Data stream) or SPMD (Single Program, Multiple Data). This is because each processor can execute different programs (or different parts of the same image processing program) on independent image streams asynchronously and concurrently, maximizing utilization and scaling.

Question 10 (Cores Optimization Problem)

Q: You're optimizing a scientific computation program and profiling reveals that 70% of the work is parallelizable while 30% remains sequential. You plan to rent a server from a cloud provider that offers machines ranging from 2 to 500 cores. You only like speed-ups in whole numbers (such as 5, 6, etc. and not fractions like 5.3 etc.). What core count should you get?

According to Amdahl's Law, the maximum theoretical speedup is:

$$S_{\max} = 1 / 0.3 = 3.333x$$

Since the speedup can never exceed 3.333x, and we prefer whole numbers, the highest possible whole-number speedup we can target is 3.

We plug $S = 3$ and $F = 0.3$ into Amdahl's law to solve for N :

$$\begin{aligned} 3 &= 1 / (0.3 + 0.7/N) \\ 0.3 + 0.7/N &= 1/3 \approx 0.3333 \\ 0.7/N &= 0.0333 = 1/30 \\ N &= 0.7 * 30 = 21 \end{aligned}$$

Therefore, you should rent a server with **21 cores**.

Final Examination, Spring 2024

Question 11 (Amdahl's Law Definition & Units)

Q: Write Amdahl's law. Clearly mention what each of the symbols in the formula is and what the units of those symbols are.

$$S = T_1 / T_p = 1 / (F + (1-F)/N)$$

- **S:** Overall Speedup (unitless ratio of execution times, i.e., seconds / seconds).
- **T₁:** Sequential execution time (seconds).
- **T_p:** Parallel execution time (seconds).
- **F:** Sequential fraction of the program (unitless fraction).
- **1-F:** Parallelizable fraction of the program (unitless fraction).
- **N:** Number of processors used (unitless count).

Since both sides of the equation are ratios/counts, they are completely unitless, and the units match perfectly on both sides.

Question 12 (Minimum Execution Time Limit)

Q: If a program needs 20 hours to complete using a single processor, but a one-hour portion of the program cannot be parallelized. What is the minimum possible execution time? Write down your assumptions.

Under Amdahl's Law, as the number of processors approaches infinity, the execution time of the parallelizable portion (19 hours) approaches 0. The serial portion (1 hour) remains fixed. Therefore, the minimum possible execution time is **1 hour**.

Assumptions: We assume zero parallelization overhead, perfect load balancing of the parallel part across cores, and that the serial portion remains strictly sequential.

Question 13 (Amdahl's Law Speedup Calculation)

Q: Assume a serial program split into four parts: $p_1 = 0.11$, $p_2 = 0.18$, $p_3 = 0.23$, and $p_4 = 0.48$. The 1st part is not sped up, 2nd is sped up 5x, 3rd is sped up 20x, and 4th is sped up 1.6x. What is the overall speedup?

Let initial sequential execution time be $T_{\text{seq}} = 1.0$.

$$T'_{\text{par}} = (0.11 / 1) + (0.18 / 5) + (0.23 / 20) + (0.48 / 1.6)$$

$$T'_{\text{par}} = 0.11 + 0.036 + 0.0115 + 0.3 = 0.4575 \text{ seconds}$$

Overall Speedup (S):

$$S = 1.0 / 0.4575 \approx 2.186$$

The overall speedup of the program is approximately **2.19**.

Final Examination, Spring 2023

Question 14 (Speed Doubling Calculation)

Q: What is Amdahl's law? According to this law, if we want to double the overall speed of a system having 60% parallelizable code, how many processors would be required?

Amdahl's law shows an upper-bound on the maximum theoretical speedup that can be achieved on a program with a fixed sequential bottleneck F .

Given: $1 - F = 0.60$ implies $F = 0.40$ (serial fraction). Desired Speedup: $S = 2$.

$$\begin{aligned} 2 &= 1 / (0.4 + 0.6/N) \\ 0.4 + 0.6/N &= 0.5 \\ 0.6/N &= 0.1 \Rightarrow N = 6 \end{aligned}$$

Exactly **6 processors** are required.

Final Examination, Spring 2022

Question 15 (Karp-Flatt Terminology & Concept)

Q: Explain the following terms: (a) Non-uniform Memory Access (NUMA), (b) Amdahl's law and its shortcomings.

(a) NUMA: A shared-memory architecture where memory access times vary depending on the memory's physical location relative to the processor. Accessing local memory is much faster than accessing remote memory.

(b) Shortcomings of Amdahl's Law:

- **Fixed Workload Assumption:** It assumes the problem size remains constant as we add more processors, whereas in practice, developers scale up workloads to use extra computing power.
- **Ignores Parallelization Overheads:** It does not account for the real-world costs of inter-processor communication, thread creation, data partitioning, and synchronization.

Question 16 (Karp-Flatt MCQ)

Q: Assume a sequential program S has an execution time of 650 seconds. Now assume a parallel variant of S takes 85.55 seconds to complete when we have 8 processors available. The Karp-Flatt metric is approximately equal to:

Achieved Speedup: $S = 650 / 85.55 \approx 7.5979$

Karp-Flatt Metric: $f = ((1/7.5979) - 1/8) / (1 - 1/8) = 0.00661 / 0.875 \approx 0.0075$

Correct option is **(d) 0.007**.

Question 17 (Speedup MCQ)

Q: Assume a sequential program S has an execution time of 400 seconds. Now assume a parallel variant of S takes 55.55 seconds to complete when we have 8 processors available. The speedup is approximately equal to:

$S = 400 / 55.55 \approx 7.2$. Correct option is **(b) 7.2**.

Midterm I, Spring 2023

Question 18 (Karp-Flatt Experimental Analysis)

Q: Assume a sequential program S has an execution time of 500 seconds. Calculate Speedups and Karp-Flatt metrics for each, interpret the values, and write your opinion.

Processors (P)	2	4	6	8
Parallel Execution Time (s)	294	203.84	176.62	164.92
Speedup (S)	1.70	2.45	2.83	3.03
K-F Metric (e)	0.18	0.21	0.22	0.23

Interpretation: Since experimentally determined serial fraction 'e' increases steadily with p, it indicates that parallelization overhead (communication, sync, etc.) is the primary contributor to poor scaling. We need to focus on reducing this overhead to achieve better speedups.

Sessional-I (Midterm I) Exam, Spring 2024

Question 19 (Flynn and Karp-Flatt Concepts)

Q: Analyze the given Karp-Flatt equation and experimental data table. What can we infer about the quality of parallelism?

- (a) s is the achieved speedup.
- (b) f is the Karp-Flatt metric representing the experimentally determined serial fraction. It is unitless.
- (c) **Inference:** The program achieves near-linear speedups as processors scale (e.g., 6.96x speedup on 8 cores). Additionally, the Karp-Flatt metric f remains extremely stable (nearly constant around 0.021) across all runs. This indicates negligible parallelization overhead. The slight drag on perfect speedup is solely due to the fixed 2.1% inherently serial part of the code itself.

Quiz 1, Spring 2023 & Spring 2025

Question 20 (Amdahl's Upgrade Options)

Q: Calculate speedup in each case and identify which option is better:

(i) Increase processors from 1 to 8, parallelize 40% of code.

(ii) Increase processors from 1 to 4, parallelize 70% of code.

Option (i): $F = 0.60$, $N = 8$. $S1 = 1 / (0.60 + 0.40/8) = 1 / 0.65 \approx 1.54x$

Option (ii): $F = 0.30$, $N = 4$. $S2 = 1 / (0.30 + 0.70/4) = 1 / 0.475 \approx 2.11x$

Conclusion: Option (ii) is significantly better because it achieves a much higher speedup ($2.11 > 1.54$).

Question 21 (Hardware Classification under Flynn's Taxonomy)

Q: Name a hardware example for each of the following four classifications under Flynn's Taxonomy: (a) SISD, (b) SIMD, (c) MIMD, (d) MISD.

- **(a) SISD:** Intel Pentium Pro (or any single-core uniprocessor).
- **(b) SIMD:** Vector supercomputers, SSE/AVX registers in Intel/AMD processors, GPUs.
- **(c) MIMD:** Multi-core CPUs (e.g., Intel Core i7, AMD Ryzen), multi-processor computers, clusters.
- **(d) MISD:** No commercial general-purpose hardware exists. Google's TPU (systolic arrays) or fault-tolerant space shuttle computers can be considered examples.

PDC Quiz 2, Spring 2024

Question 22 (Interconnection Networks)

Q: Define bisectional bandwidth of a topology. What is the bisectional bandwidth of a fully connected topology having n nodes?

Bisectional bandwidth is the minimum aggregate bandwidth of links that must be cut to divide the network topology into two equal halves.

For a fully connected (all-to-all) topology with n nodes, splitting it into two equal halves of size $n/2$ cuts:

$$\text{Cut links} = (n/2) * (n/2) = n^2 / 4$$

Therefore, the bisectional bandwidth is $n^2 / 4$ (assuming unit bandwidth per link).

=== BOOK QUESTIONS ===

Question 23 (Amdahl's Law Proof - Problem 5.1)

Q: If a problem of size W has a serial component W_S , prove that W/W_S is an upper bound on its speedup, no matter how many processing elements are used.

The serial runtime to solve a problem on a single processor is $T_{seq} = W_S + W_p = W$. When executed on p processors, the serial part W_S must still be run sequentially. Thus, the parallel runtime T_P is bounded from below by the serial execution time: $T_P \geq W_S$.

Since Speedup (S) is defined as $S = W / T_P$, substituting the inequality gives:

$$S \leq W / W_S$$

Hence, W/W_S is the absolute upper bound limit on the speedup.

Question 24 (Scaled Speedup Analysis - Problem 5.5)

Q: For the problem of adding n numbers on p processing elements, compare the standard speedup curve with the scaled speedup curve for p in $\{1, 4, 16, 64, 256\}$.

Standard speedup curve peaks at $p = 16$ (4.32x) and then declines as communication overhead starts dominating the fixed workload. The scaled speedup curve grows linearly with p , reaching 191.06x at $p = 256$, because scaling the problem size keeps the computational work per processor constant.

Question 25 (Cycles in Hypercubes - Problem 2.11)

Q: Show that there are no odd-length cycles in a d -dimensional hypercube.

Nodes in a d -dimensional hypercube can be labeled using d -bit binary strings. An edge exists between two nodes if and only if their labels differ in exactly one bit position. Let the parity of a node be the sum of its binary string modulo 2.

Since adjacent nodes differ by exactly one bit, every edge connects an even-parity node to an odd-parity node. This classifies the hypercube as a bipartite graph. In bipartite graphs, any path starting and ending at the same node must alternate between the two sets an even number of times. Therefore, all cycles must be of even length, and no odd-length cycles can exist.

Question 26 (FFT Scalability - Problem 13.1)

Q: Let the serial run time of an n -point FFT computation be $t_c * n * \log_2(n)$. Write expressions for the speedup and efficiency. What is the isoefficiency function if an efficiency of 0.6 is desired? How will it change if an efficiency of 0.4 is desired?

Speedup & Efficiency:

$$\text{Speedup } (S) = (p * \log_2(n)) / (\log_2(n) + 0.2 * \log_2(p))$$
$$\text{Efficiency } (E) = 1 / (1 + (0.2 * \log_2(p) / \log_2(n)))$$

Isoefficiency for $E = 0.6$:

$$0.6 = 1 / (1 + 0.2 * \log_2(p) / \log_2(n)) \Rightarrow n = p^{0.3}$$
$$W = 0.3 * p^{0.3} * \log_2(p) \Rightarrow \Theta(p^{0.3} * \log(p))$$

Isoefficiency for $E = 0.4$:

$$0.4 = 1 / (1 + 0.2 * \log_2(p) / \log_2(n)) \Rightarrow n = p^{0.133}$$
$$W = 0.133 * p^{0.133} * \log_2(p) \Rightarrow \Theta(p^{0.133} * \log(p))$$

As the desired efficiency decreases (from 0.6 to 0.4), the exponent of p in the isoefficiency function decreases (from 0.3 to 0.133), meaning the algorithm requires less extreme workload scaling to maintain a lower efficiency.